OOP with Java - Part 3

Statements, Expressions, Variables, Data Types & Operators

Informatics Engineering, UNRAM, 2026

What We Cover Today

- Bridge: from objects back to the fundamentals
- Statements & Expressions
- Variables, Scope & Data Types (primitive vs reference)
- Constants, Type Casting & Rounding
- Operators (arithmetic, assignment, relational, logical, bitwise)
- Comparing values: == vs .equals()
- Decision statements (if / if-else / switch)
- Reading input with Scanner
- This week's assessment exercises

Primitive vs Reference Types

Remember the copy trap from last week? Here is WHY it happens.

Primitive types hold the VALUE directly

```
int a = 5;
int b = a;   // b gets a COPY of 5
b = 10;
// a is still 5, they are independent
```

Reference types hold an ADDRESS (a pointer)

```
Student m = new Student();
Student c = m;   // c copies the ADDRESS
c.name = "Adi";
// m.name is ALSO "Adi", same object!
```

Rule:
primitive = int, double, char, boolean, byte, short, long, float (lowercase).
Reference = every class type: String, Student, Car, arrays… (Uppercase).
A primitive variable stores the value; a reference variable stores the location of an object.

⚠ **"=" copies what the variable holds. For primitives that is the value. For references that is the address, not the object. This is the single most common beginner bug in Java.**

Comparing Values: == vs .equals()

- == compares IDENTITY : "are these the same object in memory?"
- .equals() compares CONTENT : "do these have the same value?"

```
String s1 = new String("hello");
String s2 = new String("hello");

System.out.println(s1 == s2);         // false - two different objects
System.out.println(s1.equals(s2));    // true  - same content

// For strings, ALWAYS use .equals() to compare text:
if (name.equals("Adi")) { ... }              // correct
if (name.equalsIgnoreCase("adi")) { ... }    // ignore case
```

When to use which

- Primitives (int, char…): use ==
- Objects/Strings: use .equals()
- == on objects checks the address, not the text

This connects directly to the copy trap: == asks "same box?", .equals() asks "same contents?"

Strings are Immutable + Reading Input with Scanner

String Immutability

- A String's content can never be changed after it is created.
- "Modifying" a String actually creates a NEW String object.

```
String s = "Hello";
s.concat(" World");
System.out.println(s);
// still "Hello"!  concat returned
// a NEW string we ignored

s = s.concat(" World");
System.out.println(s);
// now "Hello World"
```

Reading Input - Scanner

- Scanner lets your program read what the user types.
- Import it, then read by type.

```
import java.util.Scanner;

Scanner input = new Scanner(System.in);

System.out.print("Enter your name: ");
String name = input.nextLine();

System.out.print("Enter your age: ");
int age = input.nextInt();

System.out.println("Hi " + name +", age " + age);
```

1. Statements & Expressions

The grammar of Java

Statements & Expressions

Statement

- A command that makes something happen - a single action in a Java program.
- **Every statement ends with a semicolon (;)**
- Statements are grouped with braces { } - this group is called a block.

```
int productionYear;

{
    int a = 5;
    int b = 10;    // this is a block
}
```

Expression

- A statement can produce a value - this value is the return value.
- A return value can be a number, a boolean, or an object.
- **A method with no return value uses the keyword void.**

```
quotient = a / b;

boolean status = (x > 0);

// void method: returns nothing
public void show() {
    System.out.println("hi");
}
```

Reserved Keywords

These words are reserved by Java — you cannot use them as names for your variables, methods, or classes.

abstract	continue	for	new	switch	assert
default	package	synchronized	boolean	do	if
private	this	break	double	implements	protected
throw	byte	else	import	public	throws
case	enum	instanceof	return	transient	catch
extends	int	short	try	char	final
interface	static	void	class	finally	long
strictfp	volatile	const	float	native	super
while					

2. Variables, Scope & Data Types

Where data lives

Variables : Declaring, Assigning, Naming

Declare

A variable is a named location in memory that stores a value. Its value can change anywhere in the program.

```
type name;

int product;
String a, b, c;
```

Assign

Use = to give a value. Always give local variables an initial value to prevent bugs. Instance variables get a default automatically.

```
int sum = 0;
boolean status = false;
```

Name (camelCase)

First word lowercase; capitalize the first letter of each following word.

```
int result;
boolean engineOn;
Button openFile;
```

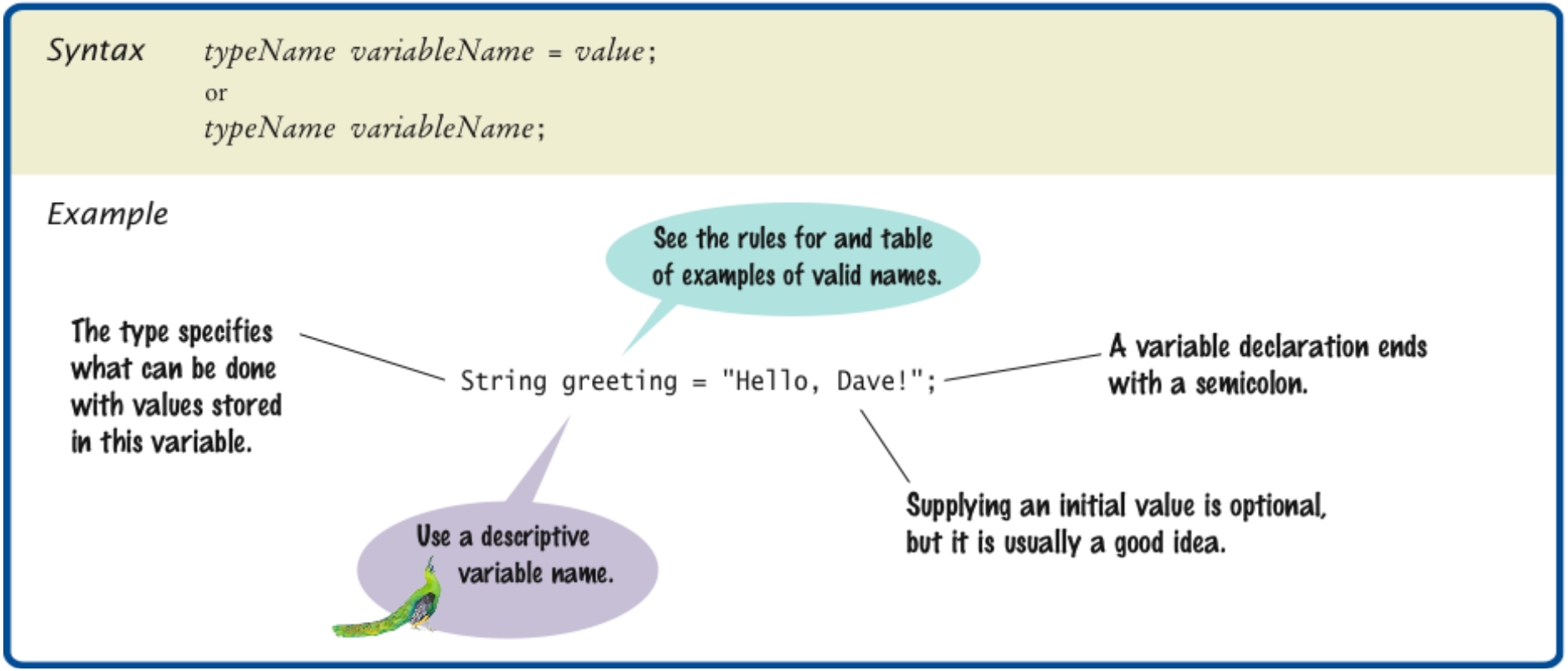
Convention: Oracle Java Code Conventions

Variabel Declaration

Syntax

typeName variableName = value;
or
typeName variableName;

Example



Variable Scope

A variable is visible only within its scope, resolved from the innermost block outward.

1. Local Variable

Declared inside a method or a smaller block; exists only there.

2. Parameter

A variable listed in a method's argument list.

3. Instance Variable

Belongs to each object : different value per object.

4. Class Variable (static)

Shared by the class and ALL its objects. Marked with the static keyword.

```
public class Bilangan {
    static int counter = 0;    // class variable
    int value;                 // instance variable

    public Bilangan(int value) {
        this.value = value;    // parameter -> instance
        counter++;             // shared across objects
    }
}
```

static vs non-static : Seeing the Difference

```
public class Bilangan {
    static int counter = 0;    // ONE shared copy
    int value;                 // each object its own

    public Bilangan(int value) {
        this.value = value;
        counter++;
    }
    public void info() {
        System.out.println("Value: "    + value);
        System.out.println("Counter: " + counter);
    }
}
```

```
Bilangan b1 = new Bilangan(50); b1.info();
Bilangan b2 = new Bilangan(15); b2.info();
Bilangan b3 = new Bilangan(30); b3.info();
```

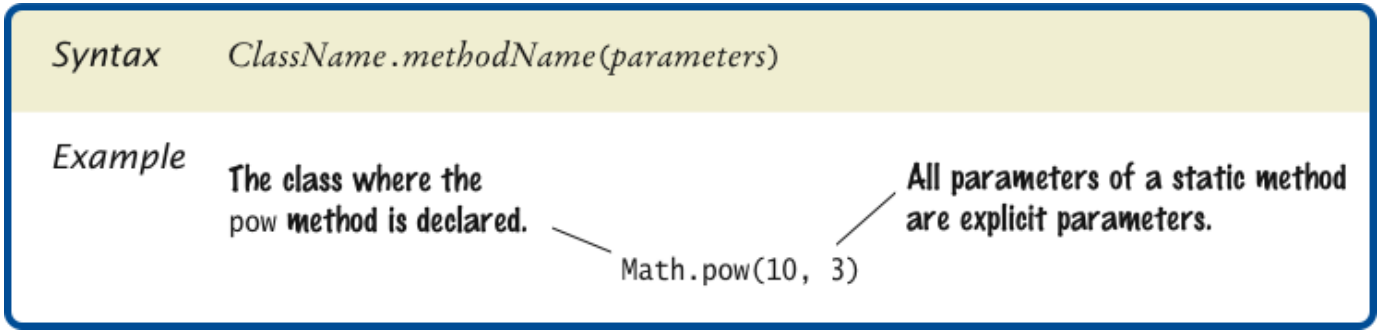
counter is static

```
Value: 50
Counter: 1
Value: 15
Counter: 2
Value: 30
Counter: 3
```

if it were non-static

```
Value: 50
Counter: 1
Value: 15
Counter: 1
Value: 30
Counter: 1
```

static = belongs to the class (one shared copy). non-static = belongs to each object (a separate copy each). A static member cannot use this, there is no single object to refer to.



Static Modifier (Method)

- Just like static variables, when a method is marked with the static modifier, it is controlled by the class rather than by an object.
- The method can be called without creating an object.
- Example of Static methods : methods that only perform mathematical calculations.
  https://docs.oracle.com/javase/8/docs/api/java/lang/Math.html

Primitive Data Types

Type	Meaning	Size	Range
`byte`	Integer	8-bit	–128 … 127
`short`	Integer	16-bit	–32,768 … 32,767
`int`	Integer	32-bit	± ~2.1 billion
`long`	Integer	64-bit	± ~9.2×10^{18}
`float`	Decimal (6–7 digits)	32-bit	± 3.4×10^{38}
`double`	Decimal (14–15 digits)	64-bit	± 1.7×10^{308}
`char`	Character (Unicode)	16-bit	\u0000 … \uffff
`boolean`	Logic (true/false)	1-bit	true / false

Four general categories

- Whole numbers
- Decimals
- Characters
- Boolean

The result of a math operation follows the type of its operands (int ÷ int = int).

```
char c1 = 88;    // → 'X' (Unicode 88)
char c2 = 'X';
System.out.println(c1); // X
```

Primitive vs Reference Data Types (Recap)

Primitive Types

- Built into the Java compiler : faster processing
- **Written in lowercase**
- Hold the value directly
- Examples: int, double, char, boolean

Reference Types (Classes)

- Classes from Java's libraries (java.lang) or your own
- **Written with a Capital first letter**
- Hold a reference (address) to an object
- Examples: String, Integer, Double, Student, Car

Every array and every object you create is a reference type, which is exactly why "=" copies the address, not the contents.

Constants, Type Casting & Rounding

Constants (final)

Use final so a value cannot change. Often combined with static on a class. Name in UPPERCASE.

```
final float PI = 3.141592f;
static final Boolean DEBUG = false;
```

Type Casting

Force a value into another type. Narrowing (double→int) loses the fraction.

```
double i = 10.56;
int forced = (int) i;
// forced = 10
// (fraction dropped)
```

Rounding

Math.round() converts a decimal to the nearest whole number.

```
int r = Math.round(13.75);
// r = 14
```

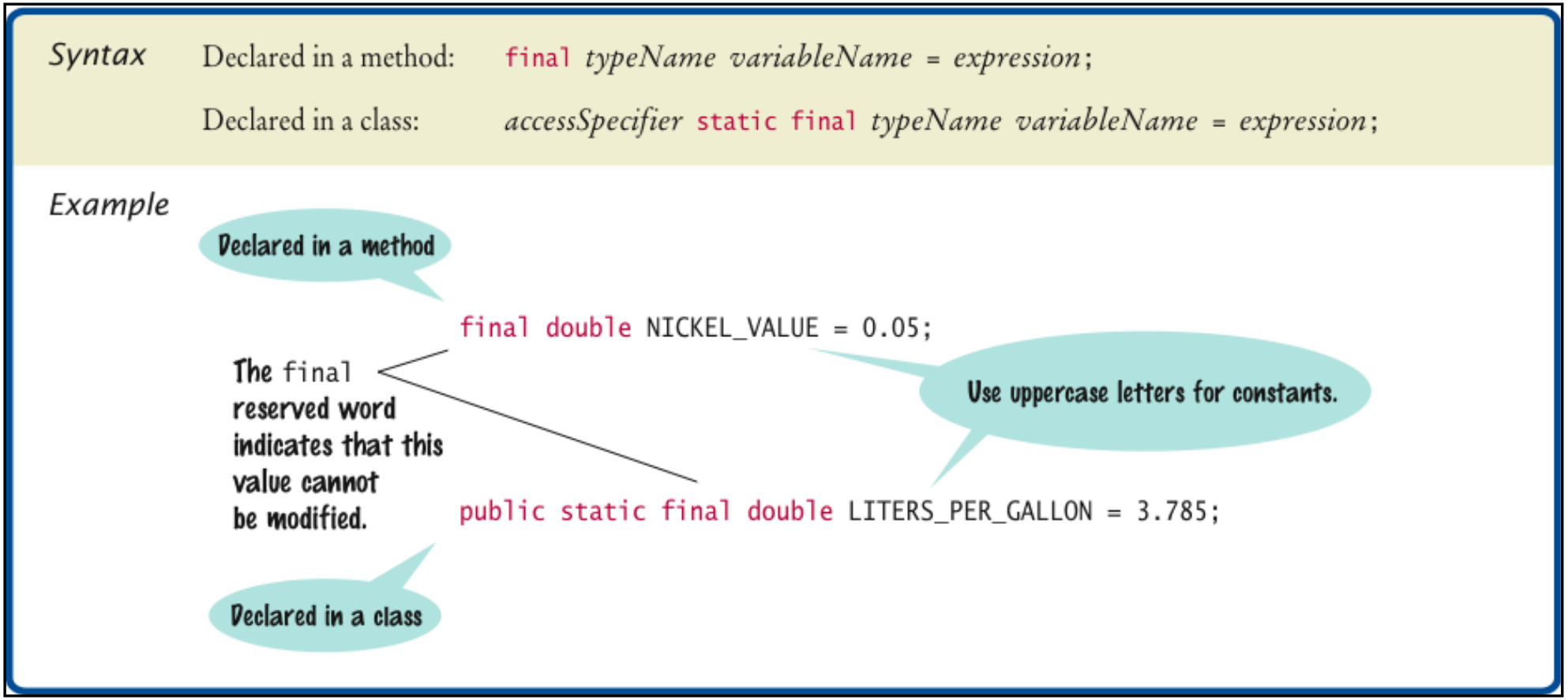
```
// Triangle area, cast to keep the decimal:
int base = 3, height = 7;
double area = (double)(base * height) / 2;    // 10.5
System.out.println("Area: " + area);
```

Syntax

Declared in a method: `final` *typeName variableName* = *expression*;

Declared in a class: *accessSpecifier* `static final` *typeName variableName* = *expression*;

Example



Constant Variable

3. Operators

Building expressions

Arithmetic & Assignment Operators

Arithmetic

Op	Meaning	Example
+	Addition	`3 + 4`
−	Subtraction	`5 - 7`
*	Multiplication	`5 * 5`
/	Division	`14 / 7`
%	Modulus (remainder)	`20 % 7`

Compound Assignment

Expr	Meaning
`x += y`	`x = x + y`
`x -= y`	`x = x - y`
`x *= y`	`x = x * y`
`x /= y`	`x = x / y`

Useful Math methods:

`Math.sqrt(x)` · `Math.pow(x,y)` · `Math.round(x)` · `Math.min(x,y)` · `Math.max(x,y)` · `Math.PI`

⚠ **int ÷ int = int: 14 / 7 = 2, but 13 / 7 = 1 (not 1.857). Cast to double to keep decimals.**

Relational & Logical Operators

Relational (return boolean)

Op	Meaning
==	Equal
!=	Not equal || < >	Less / greater than
<= >=	Less / greater or equal

Logical (combine booleans)

Op	Meaning
&&	Logical AND (both true)
||	Logical OR (either true)
!	Logical NOT (invert)

⚠ **Comparing strings: use s1.equals(s2), NEVER s1 == s2. == checks identity (same object); .equals() checks content. Use equalsIgnoreCase() to ignore case.**

```
int age = 36;
boolean r1 = age < 25;     // false
boolean r2 = age != 26;    // true
boolean ok = (age > 18) && (age < 65);    // true
```

Relational Operators

Table 1 Relational Operator Examples

Expression	Value	Comment
`3 <= 4`	`true`	3 is less than 4; <= tests for "less than or equal".
`3 =< 4`	**Error**	The "less than or equal" operator is <=, not =<, with the "less than" symbol first.
`3 > 4`	`false`	> is the opposite of <=.
`4 < 4`	`false`	The left-hand side must be strictly smaller than the right-hand side.
`4 <= 4`	`true`	Both sides are equal; <= tests for "less than or equal".
`3 == 5 - 2`	`true`	== tests for equality.
`3 != 5 - 1`	`true`	!= tests for inequality. It is true that 3 is not 5 – 1.
`3 = 6 / 2`	**Error**	Use == to test for equality.
`1.0 / 3.0 == 0.333333333`	`false`	Although the values are very close to one another, they are not exactly equal. See Common Error 4.3.
`"10" > 5`	**Error**	You cannot compare a string to a number.
`"Tomato".substring(0, 3).equals("Tom")`	`true`	Always use the `equals` method to check whether two strings have the same contents.
`"Tomato".substring(0, 3) == ("Tom")`	`false`	Never use == to compare strings; it only checks whether the strings are stored in the same location. See Common Error 5.2 on page 180.
`"Tom".equalsIgnoreCase("TOM")`	`true`	Use the `equalsIgnoreCase` method if you don't want to distinguish between uppercase and lowercase letters.

Boolean Operators

Table 3 Boolean Operators

Expression	Value	Comment
`0 < 200 && 200 < 100`	`false`	Only the first condition is true.
`0 < 200 || 200 < 100`	`true`	The first condition is true.
`0 < 200 || 100 < 200`	`true`	The `||` is not a test for "either-or". If both conditions are true, the result is true.
`0 < 100 < 200`	Syntax error	**Error:** The expression `0 < 100` is `true`, which cannot be compared against `200`.
`0 < x || x < 100`	`true`	**Error:** This condition is always true. The programmer probably intended `0 < x && x < 100`. (See Common Error 5.5).
`0 < x && x < 100 || x == -1`	`(0 < x && x < 100) || x == -1`	The `&&` operator binds more strongly than the `||` operator.
`!(0 < 200)`	`false`	`0 < 200` is `true`, therefore its negation is `false`.
`frozen == true`	`frozen`	There is no need to compare a Boolean variable with `true`.
`frozen == false`	`!frozen`	It is clearer to use `!` than to compare with `false`.

4. Decision Statements

if / if-else / switch

if / if-else / if-else if-else

```
// if
if (total >= 100000) {
    discount = total / 10;
}

// if-else
if (total >= 100000) {
    discount = total / 10;
} else {
    discount = 0;
}
```

```
// if-else if-else  (grading)
int score = 86;
char grade;

if (score >= 90)      grade = 'A';
else if (score >= 80) grade = 'B';
else if (score >= 70) grade = 'C';
else                  grade = 'D';

System.out.println("Grade = " + grade);
```

Use if-else if-else when a value falls into one of several ranges. Order matters, the first true branch wins.

switch

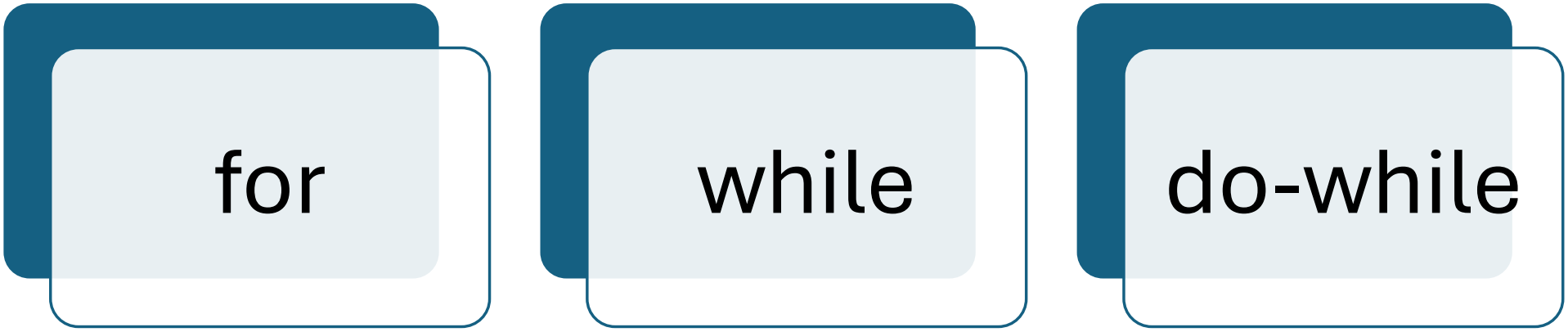
Use switch to take different actions for a set of specific values.

```
switch (expression) {
    case valueOne:
        statement1;
        break;
    case valueTwo:
        statement2;
        break;
    default:
        statementN;
}
```

```
switch (day) {
    case 1: name = "Monday";  break;
    case 2: name = "Tuesday"; break;
    case 3: name = "Wednesday"; break;
    default: name = "Other";
}
```

⚠ **Don’t forget break; without it, execution "falls through" into the next case.**

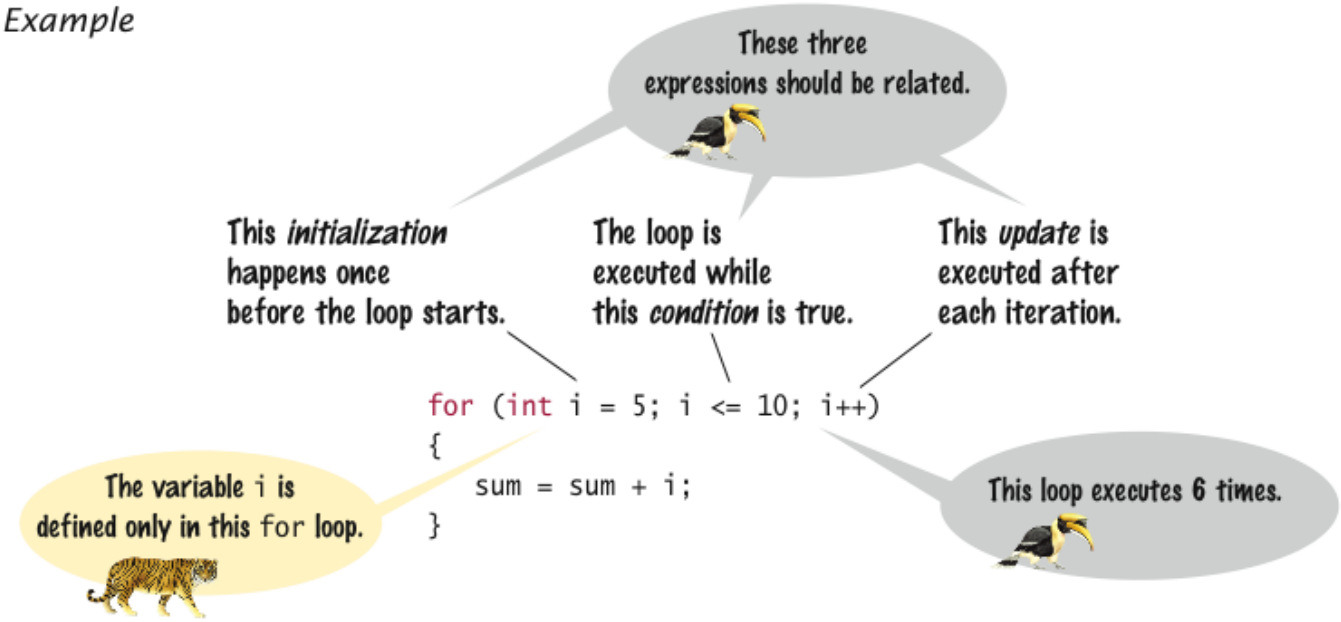
Looping



for

Syntax `for` (*initialization*; *condition*; *update*)
statement

Example



This Week's Exercises

Concept practice : case studies

Exercise A : House Tax (PBB) using OOP

Case

- Each Father (Ayah) may own at most one house.
- Each house pays land-and-building tax (PBB): type 36 → 4% of price; type 45 → 6%; type above 45 → 9%.
- Akbar, Bambang, and Charlie are three Fathers. Akbar's house is type 36, Bambang's is type 45, Charlie's is type 90.
- **Write a program to compute the PBB for Akbar, Bambang, and Charlie.**

Design first (before coding):

- Objects: House (attributes: type, price; method: calculatePBB()) and Father (attribute: a House).
- The tax rate depends on type - that decision (if-else) lives inside House.calculatePBB().
- Which is a class, which is an object? House is the blueprint; Akbar's house is an object.

Exercise A : Worked Answer

```
public class House {
    private int type;       // 36, 45, 90…
    private double price;

    public House(int type, double price) {
        this.type = type;
        this.price = price;
    }
    public double calculatePBB() {
        double rate;
        if (type == 36)      rate = 0.04;
        else if (type == 45) rate = 0.06;
        else                 rate = 0.09;   // type > 45
        return price * rate;
    }
}
```

```
public class HouseDemo {
  public static void main(String[] args) {
    House akbar   = new House(36, 300_000_000);
    House bambang = new House(45, 450_000_000);
    House charlie = new House(90, 900_000_000);

    System.out.println("Akbar PBB   = " +
        akbar.calculatePBB());
    System.out.println("Bambang PBB = " +
        bambang.calculatePBB());
    System.out.println("Charlie PBB = " +
        charlie.calculatePBB());
  }
}
```

Teaching point: the tax logic is encapsulated in House, the demo class just creates objects and asks for results. That is OOP thinking: data + behavior together.

Exercise B : Bank Account

Task

- Create a class Bank.
- Constructor parameter: balance.
- Methods: deposit(), withdraw(), getBalance().
- **Create BankDemo: set initial balance Rp 100,000 via the constructor, then run the three methods and show the process:**

```
Welcome to Bank ABC
Current balance: Rp 100000

Deposit: Rp 500000
Current balance: Rp 600000

Withdraw: Rp 150000
Current balance: Rp 450000
```

Home Practice

- **Circle Area: class Circle with hitungLuas() using PI * r * r (PI = 3.141592). In CircleDemo, print the area of r = 11.78 in three forms: decimal, integer (type-casting), and rounded (Math.round).**
- **Operators trace: predict the output of these before running them.**

```
int w = 5, x = 5, y, z;
y = 8 - x++;    // y = ?    x = ?
z = 8 - ++w;    // z = ?    w = ?
```

```
int age = 36;
boolean b1 = age < 25;
boolean b2 = age != 26;
// print b1 and b2
```

Bring your answers next week. We will compare your predictions with the real output; being wrong is useful, it shows exactly which concept to fix.